FastAPI 接口层 · 速查表

app.py · api/deps.py · api/routes.py · api/sse.py

RAG 企业知识库智能问答系统

12 路径 / 13 操作 · 线程池 40 · 心跳 15s · 队列 64 · 上传 50MB · 41 用例通过

① 应用组装 · app.py

FastAPI(title, version, lifespan=..., openapi_tags=...)

• lifespan(@asynccontextmanager): ensure_dirs → build_pipeline(strict=False) → 挂 app.state.pipeline

降级 加载失败先标记 ready=False，服务照常启动（可用性优先，不让坏索引拖死整个服务）

app.include_router(router, prefix="/api/v1")

• /health、/metrics、/、/favicon.ico 不带前缀 + include_in_schema=False → 探针可直接打，/docs 干净

中间件 CORSMiddleware 默认 ["*"]、credentials=False；@app.middleware("http") 记访问日志与耗时

流式响应的耗时只记到「开始返回」，真实端到端耗时看 done.total_ms

• 启动顺序：ensure_dirs → 加载索引并重建 BM25 → 挂到 app.state；关闭时无需清理

异常 @app.exception_handler(RagError) → JSON{error, message, detail}

状态码由异常类自带：415 类型不支持 / 422 解析失败 / 409 索引未就绪 / 502 上游失败

② 依赖注入 · api/deps.py

with _lock: # threading.Lock + 模块级 _pipeline 单例

• 防「并发首请求同时构建两份索引」的竞态；FAISS/BM25 是内存对象，绝不能每请求重建

def get_pipeline() -> Iterator[RagPipeline]: ... yield pipeline

• 生成器依赖可做请求级清理；这里统一取用与错误语义

• ready=False 且索引非空 → HTTPException(409)：宁可不服务，也不喂半残数据

pipeline_from_request(request) # /health 这类「必须能应答」的端点用它

• 收益：FAISS/BM25 只加载一次；放 app.state 又让测试能替换成已索引好的流水线

• 实例放在 app.state 而非模块全局 → 测试可注入替身（TestClient 的 with 会触发 lifespan）

③ 端点契约 · routes + schemas

• 13 个操作全带 tags / summary / response_model → /docs 里可直接交互调试

question: Field(min_length=1, max_length=1000) + field_validator 去空白

top_k ge=1 le=20 · threshold ge=0 le=1 · history max_length=20

role: Literal["user", "assistant"]

• 非法输入自动 422 并给字段级提示，无需手写校验代码

• 上传：list[UploadFile]、后缀白名单、50MB 上限、逐文件容错、finally close

• 删除：removed_chunks==0 → 404，符合 HTTP 语义

• rebuild 的服务端目录参数经 _resolve_server_path() 前缀校验，越界 400（防目录穿越）

• POST /chat/stream 是主链路（可带 history、可 AbortController 取消）；GET 版兼容原生 EventSource

④ 异步模型（最容易被追问）

路由调同步逻辑	**await run_in_threadpool(...)**
生成器里检索	**await asyncio.to_thread(...)**
上游 LLM 流式	**原生 async，不占线程**
毫秒级内存计算	**直接跑，不必进池**

线程池与并发账

令牌 anyio 默认 40 个；池满时新任务排队，事件循环不阻塞

• 同步代码写在 async def 里 = 冻住整个事件循环，连别人的 SSE 心跳都发不出去

• 慢操作走 async IO（不占线程）+ 重 CPU 进线程池（受 40 限制）→ SSE 能挂很多连接

多 worker 内存索引每个 worker 各存一份且互不同步（上传只落一个）→ 单 worker + 多副本 + 共享存储，或索引外置

⑤ SSE 实现 · api/sse.py

media_type="text/event-stream" + X-Accel-Buffering: no

Cache-Control: no-cache, no-transform + Content-Encoding: identity

• 少一个 X-Accel-Buffering，Nginx 就把流攒成一次性输出 —— 「本地快、上线慢」的元凶

queue = asyncio.Queue(maxsize=64); task = create_task(produce())

• 生产/消费解耦：才有空隙插心跳；maxsize 顺带提供背压

await asyncio.wait_for(queue.get(), timeout=15) → 超时 yield ": ping"

• 心跳是 SSE 注释行（客户端自动忽略），防网关 60s 读超时掐断长连接

• 异常也走事件：event: error —— 响应头已发出就改不了状态码，所以接口永远 200，不用 5xx

finally: task.cancel() + 吞掉 CancelledError / GeneratorExit

• 客户端断开必须取消上游，否则连接泄漏 + 白烧 token

⑥ 前端：为什么手写 SSE 而不用 EventSource

• EventSource 只能 GET、不能带请求体 → 改用 fetch + response.body.getReader()

• 按 \n\n 切帧；未段留作 buffer；跳过 ":" 开头的行；AbortController 取消

答法：SSE 是协议不是库，浏览器原生只有 GET 版，所以按协议自己解析

⑦ 追问速答

为何选 FastAPI？	**原生 async + 校验 + DI + 自动文档**
async 里写阻塞？	**事件循环被占，全站排队**
两个 to_thread？	**Starlette 封装 vs 标准库**
流式为何不返 5xx？	**响应头已发，改不了**
多 worker 怎么办？	**单 worker + 多副本 + 共享存储**
怎么测接口？	**TestClient 打 ASGI + curl -N**

口诀：组装在 lifespan · 单例靠 Lock · 阻塞进线程池 · 流式永远 200 · 断开要 cancel · 降级不崩